Variational Autoencoder

Steven Glass

March 2026

AI Declaration

The use of GenAI in this project was primarily kept to improving theoretical understanding of the VAE (via voice prompts explaining my current understanding and iteratively improving it), and clarity of writing checks (by passing a paragraph and not asking for a rewrite, but to spot any spelling errors). The bulk of the coding section of the project was done without AI assistance, but assisting with diagnosing shape errors and other minor bugs was extremely useful.

Contents

1 Introduction **2**

1.1 Latent Variable Formalism 2

1.2 Variational Inference and Deterministic Approximation 2

1.3 Traditional Autoencoders 3

1.4 Importance Sampling 4

2 Variational Autoencoder **4**

2.1 Decoder or Generative Model 4

2.2 Encoder or Latent Posterior 6

2.3 Training the VAE (putting it all together) 6

2.4 Re-parameterisation trick 7

3 Implementation **8**

3.1 Dataset 8

3.2 VAE Architectures 8

3.3 Results 9

1 Introduction

1.1 Latent Variable Formalism

In probabilistic modelling, an important distinction must be made between observable and latent random variables. Observable variables, denoted by x, correspond to quantities that may be directly measured, and are said to be governed by a probability distribution $p(x)$. In many instances, this distribution is unavailable in closed form, and instead we only have access to independent samples $x \sim p(x)$. Latent variables, denoted by z, are random variables that are hidden during the sampling procedure. While they may not be directly observable, latent variables are able to explain the underlying structure in the observable data. Our prior assumptions and knowledge about the latent variables are encoded by the prior distribution $p(z)$.

1.2 Variational Inference and Deterministic Approximation

Variational inference gets its name from the calculus of variations, which is a field of mathematics concerned with the minimisation of a functional (a mapping of a function to a scalar output) with respect to a function. Additionally, in probabilistic modelling, the task of inference is concerned with determining unknown quantities of a probabilistic model, conditioned on data.

Given some observed data x, we wish to leverage this data such that we may infer what values of the latent variable z were likely to be involved in its generation. This is done by computing the posterior distribution $p(z \mid x)$. Using Bayes rule, this posterior can be written as

$$p(z \mid x) = \frac{p(x \mid z)\, p(z)}{p(x)}. \tag{1}$$

In order to compute this posterior, one must compute the evidence $p(x)$ by marginalising over the latent variables, i.e. $p(x) = \int p(x, z) dz$. For many purposes, this integral becomes intractable very quickly due to it being over the entire latent space, meaning we cannot directly compute $p(z \mid x)$. To solve this, we must use variational inference.

We start by defining a variational distribution $q(z \mid x)$ from a family of distributions Q, which has some known structure. The objective is choose a function q that minimises the Kullback-Leibler divergence between the variational distribution q and the true underlying posterior $p(z \mid x)$. This variational optimisation task that solves for the optimal variational distribution $q^\star$ can be expressed as

$$\begin{aligned}
q^\star(z) &= \arg\min_q \ \mathrm{KL}\left(q(z \mid x) \,||\, p(z|x)\right) && (2)\\
&= \arg\min_q \left\{ \langle \log q(z \mid x) \rangle_{q(z \mid x)} - \langle \log p(z \mid x) \rangle_{q(z \mid x)} \right\} && (3)\\
&= \left\{ \langle \log q(z \mid x) \rangle_{q(z \mid x)} - \langle \log p(x, z) \rangle_{q(z \mid x)} + \langle \log p(x) \rangle_{q(z \mid x)} \right\} && (4)\\
&= \left\{ \langle \log q(z \mid x) \rangle_{q(z \mid x)} - \langle \log p(x, z) \rangle_{q(z \mid x)} + \log p(x) \right\}. && (5)
\end{aligned}$$

Because $\log p(x)$ does not depend on the variational distribution q, it can be ignored for the optimisation. The optimisation can be reframed as a maximisation problem if we take the negative of the argument, therefore the optimisation becomes

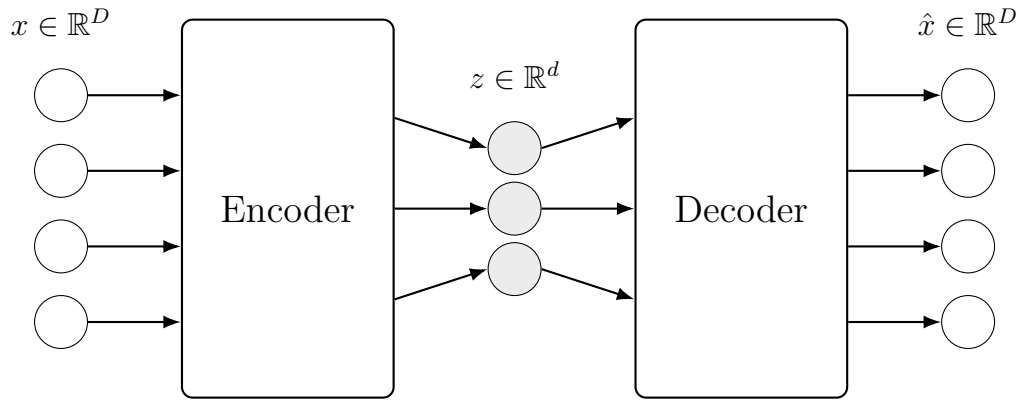

Figure 1: Visualisation of the standard autoencoder architecture, in which inputs (x) are mapped to a latent representation z using the encoder, and then the transformation back to the reconstructions ($\hat{x}$) using the decoder

$$q^\star(z) = \arg\max_q \left\{ \langle \langle \log p(x,z) \rangle_{q(z\,|\,x)} - \log q(z\,|\,x) \rangle_{q(z\,|\,x)} \right\} = \arg\max_q \{\mathcal{L}(q)\} \tag{6}$$

The functional $\mathcal{L}(q)$ is known as the Evidence Lower Bound (ELBO), which is due to the fact that it lower bounds the marginal probability of our data $p(x)$

$$\mathrm{KL}\left(q(z\,|\,x)\,||\,p(z|x)\right) = \langle \log q(z\,|\,x) \rangle_{q(z\,|\,x)} - \langle \log p(x,z) \rangle_{q(z\,|\,x)} + \log p(x) \geq 0, \tag{7}$$

Therefore

$$p(x) \geq \langle \log p(x,z) \rangle_{q(z\,|\,x)} - \langle \log q(z\,|\,x) \rangle_{q(z\,|\,x)}. \tag{8}$$

By choosing the variational posterior $q(z\,|\,x)$ restricted to family Q that maximises the ELBO, we achieve the best approximation of the true posterior $p(z\,|\,x)$ in terms of the minimisation of the KL-divergence.

1.3 Traditional Autoencoders

An autoencoder is a form of unsupervised Artificial Neural Network (ANN) that learns to encode high dimensional input data into a lower dimensional latent space using a Fully Connected Neural Network (FCN), and then learn to decode the latent representation back into the original input data using another FCN. Figure 1 depicts this architecture visually.

The encoder neural network maps inputs $x \in \mathbb{R}^D$ to lower-dimensional latent representation $z \in \mathbb{R}^d$, such that $d < D$. The latent representation is then passed to the decoder neural network as input, which maps them towards a reconstruction $\hat{x} \in \mathbb{R}^D$ of the original input data. Both encoder and decoder networks can be thought of as comprising individual FCNs that together learn the identity mapping of input data x. The network learns by minimising the loss function $L = \|x - \hat{x}\|^2$ over empirical data observations.

1.4 Importance Sampling

Importance sampling is a Monte Carlo sampling method whose results will be crucial in later deriving the training objective of the VAE. Let X be a random variable denoting our data, and suppose that X follows the distribution $p(x)$, i.e. $x \sim p(x)$. Suppose we are interested in computing the expectation of some function g applied to our random variable, which can be written as

$$\langle g(x) \rangle_{p(x)} = \int g(x)\, p(x)\, dx. \tag{9}$$

As a result of the law of large numbers, if we generate N independent and identical samples of x such that $x_i \sim p(x),$ for $i = 1, 2, ..., N$, we can approximate this integral as

$$\langle g(x) \rangle_{p(x)} \approx \frac{1}{N} \sum_{i=1}^{N} g(x_i). \tag{10}$$

This is the core idea behind Monte Carlo methods, which allows difficult integrals to be approximated using samples drawn from a probability distribution. In practice, however, two difficulties may arise. First, it may not be possible to sample directly from the target distribution $p(x)$. Second, even when sampling from $p(x)$ is possible, the Monte Carlo estimator in Eq. 10 may exhibit high variance if regions that contribute most to the integral—where the integrand $g(x)$ is large—have low probability under $p(x)$. In such cases, many samples contribute little to the estimate, resulting in inefficient and high-variance approximations.

Importance sampling solves this problem, by introducing a candidate distribution $\phi(x)$ to be the distribution we sample from, instead of $p(x)$. We manipulate the integral from Eq. 9 as follows

$$\langle g(x) \rangle_{p(x)} = \int \frac{g(x)\, p(x)}{\phi(x)}\, \phi(x)\, dx = \left\langle \frac{g(x)\, p(x)}{\phi(x)} \right\rangle_{\phi(x)}. \tag{11}$$

By Eq. 11 it is clear that the same integral can be approximated by instead sampling the data according to $\phi(x)$, and then computing the empirical average over $g(x)\, p(x)/\phi(x)$. In order to have a lower variance estimate, it is important to choose ϕ such that it is as similar as possible to the integrand $g(x)\, p(x)$.

2 Variational Autoencoder

The Variational Autoencoder (VAE) is a generative model that extends upon the traditional Autoencoder architecture by learning a probabilistic latent representation of data, where the encoder network maps inputs to a distribution over latent states, instead of a single deterministic value as in traditional autoencoders. In their seminal paper "Auto-Encoding Variational Bayes" [2], Kingma and Welling first introduce the VAE framework, and their innovation of the re-parameterisation trick allowed for the training of VAEs using backpropagation.

2.1 Decoder or Generative Model

We start by taking a look at the decoder architecture, which is designed to transform latent points z sampled from the prior distribution $p(z)$, into a parameterisation θ of the conditional distribution $p(x \mid z)$, from which new data points can be generated.

Let $p_\theta(x)$ denote the entire generative model that we wish to learn, and let $p(x)$ denote the true underlying distribution that the data was truly generated by. Although we do not know the underlying distribution $p(x)$, the training objective can theoretically be expressed in terms of it by the minimisation of the following KL-divergence

$$\theta = \arg\min_\theta \left\{ \mathrm{KL}(p(x) \,||\, p_\theta(x)) \right\}. \tag{12}$$

By simplifying the KL-divergence term further, we obtain

$$\begin{aligned} \mathrm{KL}(p(x) \,||\, p_\theta(x)) &= \int p(x) \, \log \left(\frac{p(x)}{p_\theta(x)} \right) dx \\ &= \int \log p(x) \, p(x) dx - \int \log(p_\theta(x)) \, p(x) dx. \end{aligned}$$

The optimisation task is choosing the values of the parameters θ that minimise this KL-divergence. The first integral term is a constant with respect to the parameters θ, and can therefore be ignored in the optimisation task. Therefore, the optimisation can be expressed as a maximisation task of the negative KL-divergence

$$\theta^\star = \arg\max_\theta \int \log(p_\theta(x)) p(x) dx = \arg\max_\theta \left\langle \log(p_\theta(x)) \right\rangle_{p(x)}, \tag{13}$$

where $\theta^\star$ are the optimal set of parameters. Intuitively, we are minimising the cross-entropy between the true distribution and our parameterised approximation of it. Because this integral is over the very high dimensional space x, it is computationally intractable to calculate it exactly, and therefore we must use stochastic approximation techniques. As seen previously in Eq. 10, complicated integrals can be approximated by performing an empirical average by replacing the expectation with respect to a probability distribution with an empirical average over samples drawn from that distribution.

Suppose we sample $(x_i)_{i=1}^N$ independently and from the identical underlying distribution $p(x)$, then the integral in the optimisation problem in Eq. 13 can be approximated as

$$\left\langle \log(p_\theta(x)) \right\rangle_{p(x)} \approx \frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i). \tag{14}$$

In order to compute the marginal probability $p_\theta(x)$, we need to marginalise over all the possible latent factors z that could have been responsible for generating it. This can be expressed as

$$p_\theta(x) = \int p(x \,|z) \, p(z) dz, \tag{15}$$

which may be another high dimensional integral to be approximated using Monte Carlo methods as was done previously using Eq. 10. Given we simulate values of the latent factor $(z_k)_{k=1}^K$ independently and drawn identically from the prior as $z \sim p(z)$, this approximation can be written as

$$p_\theta(x) \approx \frac{1}{K} \sum_{k=1}^{K} p_\theta(x \,|z_k) \tag{16}$$

However, before blindly trying to approximate this integral, one has to ask: "How do we choose a prior distribution over the latent z, such that it isn't just sampling irrelevant values with respect to our problem?" Because the latent z may be high-dimensional, if we try to approximate this integral by blindly sampling values of the latent factor, we never average over regions of high importance. This will make the variance of this approximation so large, that it would ultimately become meaningless. We therefore do not try to compute this approximation, and instead derive a more sophisticated training method using the other half of the VAE, the encoder.

2.2 Encoder or Latent Posterior

A traditional encoder model, as seen in Section 1.3 is a FCN trained to compress input data $x \in \mathbb{R}^D$ into a dense latent representation $z \in \mathbb{R}^d$ where $d < D$, such that when passing z to the decoder, it is able to approximate the original x as best as possible. The encoder architecture of the VAE differs, however, and instead learns to approximate the distribution $p(z \mid x)$ using variational approximation, which ultimately solves the problem we encountered at the end of Section 2.1.

We start by defining a variational distribution $q_\phi(z \mid x)$, parameterised by ϕ. The encoder of a VAE is defined by taking as input the data points $x \in \mathbb{R}^D$, and outputting the parameterisation ϕ of the distribution q.

Suppose for now that the encoder has learnt how to approximate $p(z \mid x)$ very well via the variational distribution $q_\phi(z \mid x)$, then by sampling values of z from it, we are effectively sampling latent variables that are likely under the posterior given x. Therefore, when performing the stochastic approximation seen in Eq. 16, imagine if instead of blindly sampling z from some arbitrary prior distribution, we instead sample from $q_\phi(z \mid x)$, and focus on samples informationally relevant to the data point x. Using the importance sampling theory from Section 1.4, the approximation of the integral in Eq. 16 can be rewritten as

$$p_\theta(x) \approx \frac{1}{K} \sum_{k=1}^{K} \frac{p(x \mid z_k)\, p(z_k)}{q_\phi(z_k \mid x)}. \tag{17}$$

Within the perspective of importance sampling, $q_\phi(z \mid x)$ is the proposal distribution, which allows us to focus our sampling in regions of latent space that are important in explaining the data point x.

The only remaining question is "how do we train the encoder to output the parameterisation ϕ of a sensible approximation $q_\phi(z \mid x)$ and the intractable conditional $p_\theta(z \mid x)$?" To answer this question, we will need to apply our theory of variational approximation.

2.3 Training the VAE (putting it all together)

In Section 1.2 we showed that by choosing a variational distribution that maximises the ELBO (Eq. 6), we are choosing the distribution that minimises the KL-divergence to the true posterior distribution. The ELBO of the VAE, can be reframed from Eq. 8 into

$$\mathcal{L} = \langle \log p_\theta(x, z) \rangle_{q_\phi(z \mid x)} - \langle \log q_\phi(z \mid x) \rangle_{q_\phi(z \mid x)}, \tag{18}$$

which by rewriting the joint term $p_\theta(x, z)$ into the product of the conditional distribution $p_\theta(x \mid z)$ which was is parameterised by the decoder, and the prior $p(z)$, we get

$$\mathcal{L} = \langle \log p_\theta(x \,|\, z) \rangle_{q_\phi(z \,|\, x)} - \left\langle \log \frac{q_\phi(z \,|\, x)}{p(z)} \right\rangle_{q_\phi(z \,|\, x)} . \tag{19}$$

The first term in Eq. 19 can be interpreted as the expected log-likelihood of reconstructing the observed data from our variational posterior $q_\phi(z \,|\, x)$. This term rewards the model for making realistic predictions. The second term is just the negative KL-divergence $\mathrm{KL}(q_\phi(z \,|\, x) \,||\, p(z))$, and during training enforces that the variational posterior does not drift too far from the prior distribution. Therefore, because we wish to maximise the ELBO, the loss function the VAE is trained on is just the negative ELBO.

2.4 Re-parameterisation trick

When training any neural network, a procedure known as backpropagation treats the neural network as a computational graph, and computes the derivative of the loss-function with respect to the parameters by sequentially computing simple derivatives and then multiplying them together via the chain rule. We seek the gradient of the ELBO loss with respect to θ and ϕ. The KL-term in the loss is accessible in closed form due to choosing a Gaussian posterior, and was shown by Odaibo [6] to be

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big) = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right) . \tag{20}$$

The gradient of this with respect to ϕ can therefore be easily accessed. The more complicated term to find gradients for is the reconstruction-term in the ELBO. The gradient with respect to θ was shown by Kingma and Welling [1] to simplify into

$$\nabla_\theta \, \mathcal{L} = \langle \nabla_\theta \log p_\theta(x \,|\, z) \rangle_{q_\phi(z \,|\, x)} . \tag{21}$$

If a sensible form of $p_\theta(x \,|\, z)$ is chosen, the gradient will have a closed form, and can be approximated using a Monte Carlo estimator. The more challenging gradient to compute is with respect to ϕ, as it was shown by Kingma and Welling that because the expectation is taken with respect to ϕ, we cannot simply move the gradient into the expectation. More concretely,

$$\begin{aligned} \nabla_\phi \, \mathcal{L} &= \nabla_\phi \, \langle \log p_\theta(x, z) - \log q_\phi(z \mid x) \rangle_{q_\phi(z|x)} \\ &\neq \langle \nabla_\phi \, (\log p_\theta(x, z) - \log q_\phi(z \mid x)) \rangle_{q_\phi(z|x)} . \end{aligned}$$

In their seminal paper [2], Kingma and Welling introduce the "change of variables" method, that reformulates the random sample $z = \mathcal{N}(\mu, \sigma)$ as the transformation of a random noise variable $\epsilon = \mathcal{N}(0, \mathbb{I})$ as

$$z = \sigma_\phi(x) \odot \epsilon + \mu_\phi(x). \tag{22}$$

By expressing z in terms of a transformation of ϵ, we may now take the derivative of the ELBO with respect to ϕ as

$$\nabla_\phi \langle \log p_\theta(x \,|\, z) \rangle_{q_\phi(z|x)} = \nabla_\phi \langle \log p_\theta\big(x \,|\, \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon\big) \rangle_{\epsilon \sim \mathcal{N}(0, \mathbb{I})} \tag{23}$$

$$= \langle \nabla_\phi \log p_\theta\big(x \,|\, \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon\big) \rangle_{\epsilon \sim \mathcal{N}(0, \mathbb{I})}. \tag{24}$$

This isolates the randomness in the sampling process to ϵ, and makes the ϕ dependence fully differentiable.

3 Implementation

3.1 Dataset

When testing the capabilities of the VAE on a real world problem, it is important to choose a dataset that is aligned with the strengths of the model. VAEs are designed to be able to handle high-dimensional input, and be able to learn a low-dimensional latent representation. The dataset chosen for this project is the MNIST dataset of hand-drawn digits from 0-9 [3]. The images come from a high dimensional space $\mathbb{R}^{28\times 28}$, and the VAE is trained on the set of 60 000 images, of roughly 6 000 images per digit. Each image is grayscale, and upon processing the images the value of each pixel within the image is constrained to $[0\,,1]$.

The choice of dataset is justified by each image being in a high enough dimension for it to be analytically intractable to model the distribution $p_\theta(x)$ directly (as one would need to compute a partition function Z by integrating over the entire data space), while still remaining computationally tractable in training the VAE on my local hardware. Additionally, it is intuitive that each image of a digit has a lower dimensional representation than the entire image space, as all images of the same digit fundamentally represent the same piece of information, regardless of slight pixel discrepancy. For example, consider 10 separate images of the digit 7, each may have slightly different angles users draw between the two straight lines, but they represent the same idea of the number 7, or 0111 in binary. Therefore we should hopefully be able to find a mapping from the high dimensional $\mathbb{R}^{28\times 28}$ image space, into a much lower-dimensional but structured latent-space from which we may sample in and generate new images of digits. For this project, I decided to choose a latent space to be 10-dimensional.

3.2 VAE Architectures

I investigated two distinct neural network architectures for the VAE, and in addition a single loss function modification in order to try combat posterior collapse. The entire VAE architecture was implemented via `Tensorflow.Keras` [4], using a custom training architecture.

The prior distribution $p(z)$ was chosen to be the 10-dimensional isotropic Gaussian distribution, i.e. $p(z) = \mathcal{N}(\mathbf{0}, \mathbb{I}_{10})$. The variational distribution $q_\phi(z\,|\,x)$ must therefore also be a 10-dimensional Gaussian distribution, and the encoder must therefore learn to map the input x to the parameters $\phi = \{\mu, \Sigma\}$ of this distribution. As a simplification, the covariance matrix Σ is a diagonal matrix, which greatly reduces the number of parameters needed to model. The conditional likelihood term $p_\theta(x\,|\,z)$ is chosen as the Bernoulli distribution $\text{Bernoulli}\big(x_j \,\big|\, \hat{x}_j(z)\big)$ over all the image pixels. By initially constricting $x_j \in [0, 1]$, we interpret this as a "probability pixel j is on", and by reconstructing the image and evaluating the Bernoulli probability, we quantify how likely the original data is given our reconstruction. This greatly simplifies the model, as the parameterisation θ is therefore contained purely in the reconstructed image, i.e. $\theta = \{\hat{x}_j\,,\ j = 1, 2, \ldots, (28 \times 28)\}$.

The first architecture used to test the VAE was implementing a fully connected neural network to model both the latent posterior $q_\phi(z\,|\,x)$ and the conditional likelihood $p_\theta(x\,|\,z)$. The encoder network consists of three hidden layers of size 256, 128, 128 respectively, with an output layer of size 20, the first 10 represent μ, and the following 10 represent the diagonal of σ. The decoder network also consists of three hidden layers of size 128, 128, 256 respectively, and a final output layer of size $28 \times 28 = 784$ representing the flattened reconstruction of an image.

The then implemented a separate architecture by replacing the first two hidden layers of the encoder network with convolutional layers. As the name suggests, Convolutional Neural Networks (CNNs) consist of layers that perform discrete convolutions, and are particularly effective at capturing spatial structure in images. The two convolutional layers are comprised of $32, 64$ filters respectively, and each have a 3×3 kernel, with same padding and stride of 2.

VAE Method	Recon Loss	KL Loss	Total Loss
Fully Connected Layers	84.7475	16.8205	101.5300
Convolutional Layers	81.4763	18.0804	99.5567
Convolutional Layers & β tuning	78.8407	19.0787	97.9196

Table 1: The final values of the KL-loss, reconstruction-loss, and total-loss for each VAE model investigated. The final loss is calculated over the entire training set, and is measured after 100-training epochs.

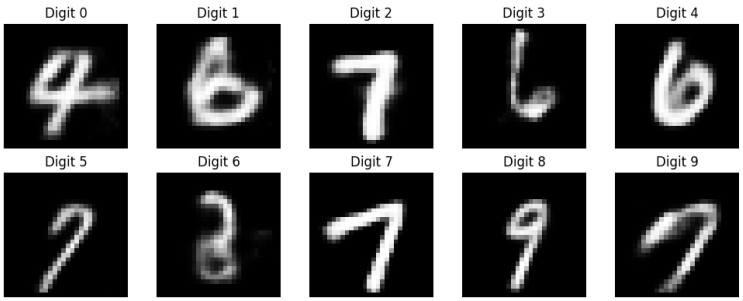

Figure 2: 10 randomly generated images using FCN VAE.

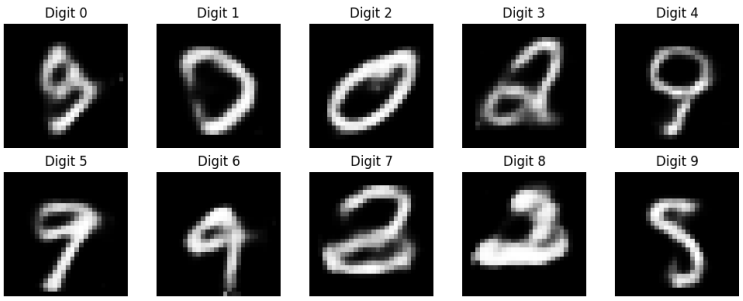

Figure 3: 10 randomly generated images using CNN VAE.

Lastly, while training the VAE, a common problem I encountered was that the KL-loss term seen in Eq. 19 was shrunk to small values very early while training. Although we do want the distribution $q_\phi(z \mid x)$ over latent space to be similar to the prior $p(z)$ upon sampling new values, if this happens too early while training we never sample important points in latent space, which completely defeats the point of introducing the encoder in the first place. Therefore, a hyper-parameter β was introduced to perform annealing of the KL-loss term. This value is initialised at $\beta = 0$, and at each epoch is computed as the index of the current epoch divided by the total number of epochs we run the annealing for. The β-annealing is performed for the first 10-epochs, and then the model is trained to optimise the loss function with $\beta = 1$ for the rest of training.

3.3 Results

As can be seen in Table 1, the VAE architecture consisting of only fully connected layers achieves the worst total-loss as despite the KL-loss being the lowest of the three methods, the reconstruction-loss is much higher. Upon introducing the convolutional layers to the encoder, I was able to slightly decrease the reconstruction loss, however, the best performing method in terms of total-loss was the convolutional model with β-annealing. It is likely that by allowing the model to focus less on matching the variational distribution to the prior during the first 10-epochs, it was able to be more flexible in what regions it was sampling the latent z from, allowing it to drive the reconstruction loss down early.

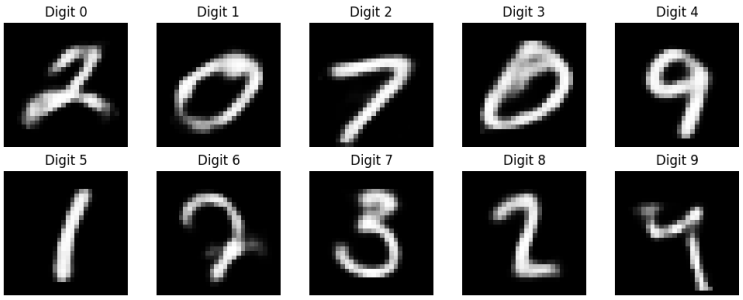

Figure 4: 10 randomly generated images using CNN VAE with β annealing.

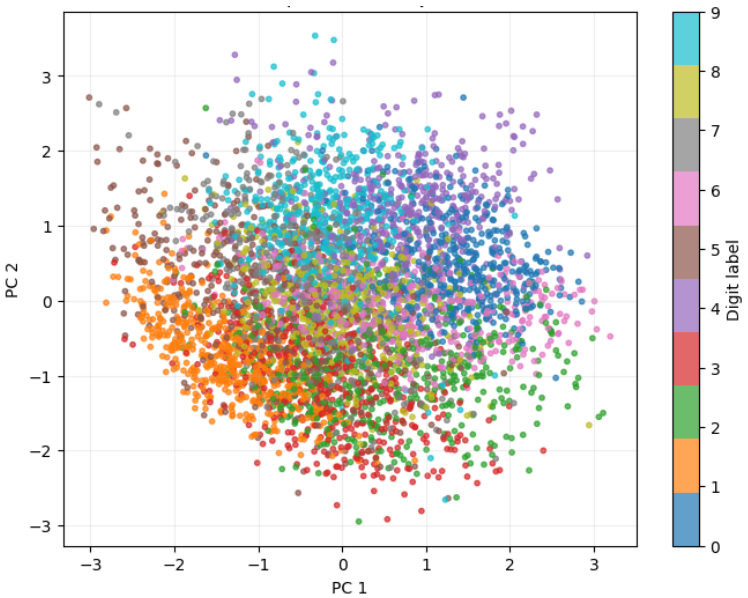

Figure 5: A PCA projection from the 10-dimensional latent space into a 2-dimensional space. For 5000 images from the testing set, the value μ encoding the mean of the distribution $q_\phi(z \mid x)$ is projected using PCA into 2-dimensions, and coloured based on what digit it represents.

In Figures 2, 3 and 4, we can get a glimpse into each method's actual ability to sample brand new images of hand-drawn digits. Purely from a visual inspection, it seems that the results found in Table 1 are reflected in the clarity of the images produced by each method, as it seems the VAE with only fully connected layers seems to produce the worst images, with an improving visual clarity for the CNN and β-annealing models.

Figure 5 visualises the distribution of points in latent space by passing 5000 unseen testing images through the encoder, computing the μ of the variational distribution $q_\phi(z \mid x)$, and performing PCA to project the data into 2-dimensions. The plot is able to visualise if the model has learnt to cluster any like digits together on unseen examples. While there appears to be some resemblance of clusters forming between the different digits, there appears to be a large overlap between the different clusters, which may likely explain why we occasionally sample new images that appear to be a fusion of two separate digits as in Figure 2.

Lastly, in Figure 6 we investigate both the distribution of the KL-divergence between the latent posterior of the best performing β-annealing CNN-model and the prior ($\mathrm{KL}(q_\phi(z \mid x) \,||\, p(z))$) over all the testing images. We see that on average, the KL-divergence far exceeds the value found on the training set in Table 1. Additionally, in Figure 6 we investigate the KL-divergence per latent dimension, which can be interpreted as each dimension's deviation from the prior, and therefore how much information it encodes. We observe that most the latent dimensions contribute an equal amount towards reconstructions.

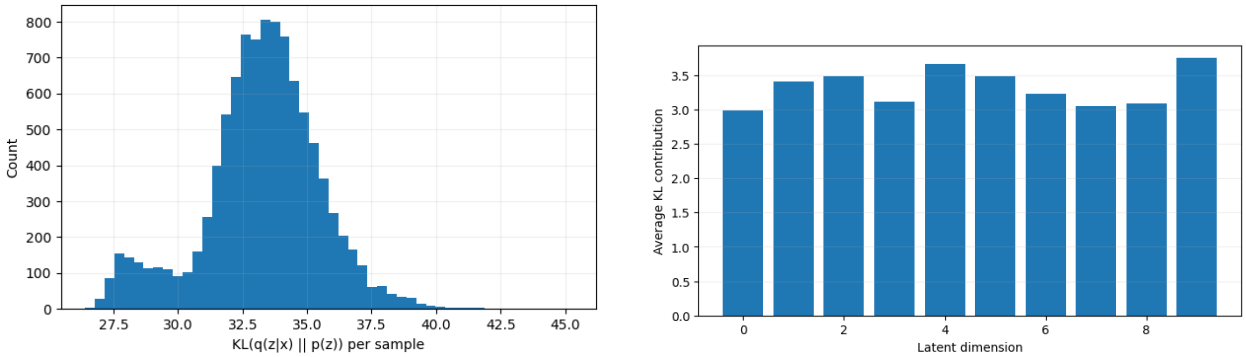

Figure 6: On the left, a histogram displaying the distribution of the KL-divergence of the latent posterior of the β-annealing CNN model and the prior over all the training samples. On the right, the average KL-divergence per dimension of the latent space over all the training samples.

4 Conclusion and Discussion

The VAE is a powerful generative model that once trained is able to generate samples of data from highly complex and high dimensional distributions, such as images. A concept that bears a rather close resemblance is that of the Probability Integral Transform (PIT), which postulates that by simply sampling random Uniform$[0, 1]$ variates, one can sample any random variable as long as we have access to its inverse cumulative density function. The VAE, however, is able to implicitly learn an inverse mapping using a neural network, making it able to sample from extremely abstract and complex distributions such as "images of cats", instead of just the standard mathematical distributions.

In the future, I would be eager to explore further the application of the VAE to a more complex data distribution. The Dsprites dataset [5] would make for a fascinating project, as the latent information is embedded into the images directly, and would be an interesting challenge to try and train a VAE to learn this information itself. Additionally, a more scientific analysis on the accuracy of generated images is needed, by quantitatively comparing generated images to the original data samples of the digit they most closely resemble.

References

[1] P. Kingma Diederik and Welling Max. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, November 2019.

[2] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[3] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[4] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015.

[5] Loic Matthey, Irina Higgins, Demis Hassabis, and Alexander Lerchner. dsprites: Disentanglement testing sprites dataset. https://github.com/deepmind/dsprites-dataset/, 2017.

[6] Stephen Odaibo. Tutorial: Deriving the standard variational autoencoder (vae) loss function, 2019.